

R-Control IP4 – MQTT Schnittstelle (Home Assistant / Generic)

Was ist das?

Über MQTT bindet sich das Gerät in eine Hausautomation ein — vor allem in **Home Assistant**. Das Gerät meldet seinen Zustand an einen MQTT-Broker und nimmt von dort Schaltbefehle entgegen.

Der große Vorteil gegenüber allen anderen Schnittstellen: Home Assistant **legt die Geräte selbst an**. Sie müssen nichts von Hand konfigurieren — nach dem Einschalten erscheinen vier Schalter, ein Temperatursensor und die Taster als Auslöser für Automatisierungen.

Sie brauchen dafür einen MQTT-Broker im Netz (z. B. das Mosquitto-Add-on in Home Assistant).

In 5 Minuten zum Ziel

Schritt 1: Broker-Daten eintragen

Weboberfläche → **Einstellungen** → **Schnittstellen** → **Home Assistant MQTT**

Feld	Beispiel
Home Assistant MQTT aktivieren	Haken setzen
MQTT Broker URI	<code>mqtt://192.168.0.50:1883</code>
MQTT Benutzername	<code>homeassistant</code>
MQTT Passwort	Ihr Broker-Passwort
Base Topic	<code>r-control-ip4</code> (Standard)
Discovery Prefix	<code>homeassistant</code> (Standard)

Schritt 2: Verbindung prüfen

Drücken Sie **Verbindung testen**. Die Statuszeile sagt sofort, ob Adresse und Zugangsdaten stimmen — noch bevor Sie speichern.

Schritt 3: Speichern

Nach dem Speichern verbindet sich das Gerät. Unter **Übersicht** → **Gerätestatus** → **Details** steht in der Zeile *Home Assistant MQTT*, ob die Verbindung steht.

Schritt 4: In Home Assistant nachsehen

Einstellungen → **Geräte & Dienste** → **MQTT** → das Gerät erscheint automatisch mit allen Entitäten. Fertig.

Verbindung klappt nicht? Der Test nennt die Ursache im Klartext, z. B. „Benutzername oder Passwort falsch“ oder „Broker nicht verfügbar“. Sie können die Einstellungen trotzdem speichern — das Gerät

verbindet sich selbstständig, sobald der Broker erreichbar ist.

Was in Home Assistant erscheint

Entität	Was Sie damit tun
4 × Schalter	Ausgänge ein- und ausschalten
1 × Temperatursensor	aktuelle Temperatur (wenn NTC angeschlossen)
4 × Geräteauslöser	auf Tastendrucke reagieren (Klick, Doppelklick, langes Drücken)

Ein Beispiel für eine Automatisierung, die auf einen Doppelklick an Eingang 1 reagiert, legen Sie in Home Assistant am einfachsten grafisch an: **Automatisierung** → **Auslöser** → **Gerät** → **R - Control IP 4** → **Eingang 1 doppelt geklickt**.

Ab hier folgt der **vollständige Referenzteil** mit allen Details.

Was ist das?

Über MQTT bindet sich das Gerät in eine Hausautomation ein — vor allem in **Home Assistant**. Das Gerät meldet seinen Zustand an einen MQTT-Broker und nimmt von dort Schaltbefehle entgegen.

Der große Vorteil gegenüber allen anderen Schnittstellen: Home Assistant **legt die Geräte selbst an**. Sie müssen nichts von Hand konfigurieren — nach dem Einschalten erscheinen vier Schalter, ein Temperatursensor und die Taster als Auslöser für Automatisierungen.

Sie brauchen dafür einen MQTT-Broker im Netz (z. B. das Mosquitto-Add-on in Home Assistant).

In 5 Minuten zum Ziel

Schritt 1: Broker-Daten eintragen

Weboberfläche → **Einstellungen** → **Schnittstellen** → **Home Assistant MQTT**

Feld	Beispiel
Home Assistant MQTT aktivieren	Haken setzen
MQTT Broker URI	<code>mqtt://192.168.0.50:1883</code>
MQTT Benutzername	<code>homeassistant</code>
MQTT Passwort	Ihr Broker-Passwort
Base Topic	<code>r-control-ip4</code> (Standard)
Discovery Prefix	<code>homeassistant</code> (Standard)

Schritt 2: Verbindung prüfen

Drücken Sie **Verbindung testen**. Die Statuszeile sagt sofort, ob Adresse und Zugangsdaten stimmen — noch bevor Sie speichern.

Schritt 3: Speichern

Nach dem Speichern verbindet sich das Gerät. Unter **Übersicht** → **Gerätestatus** → **Details** steht in der Zeile *Home Assistant MQTT*, ob die Verbindung steht.

Schritt 4: In Home Assistant nachsehen

Einstellungen → **Geräte & Dienste** → **MQTT** → das Gerät erscheint automatisch mit allen Entitäten. Fertig.

Verbindung klappt nicht? Der Test nennt die Ursache im Klartext, z. B. „Benutzername oder Passwort falsch" oder „Broker nicht verfügbar". Sie können die Einstellungen trotzdem speichern — das Gerät verbindet sich selbstständig, sobald der Broker erreichbar ist.

Was in Home Assistant erscheint

Entität	Was Sie damit tun
4 × Schalter	Ausgänge ein- und ausschalten
1 × Temperatursensor	aktuelle Temperatur (wenn NTC angeschlossen)
4 × Geräteauslöser	auf Tastendrucke reagieren (Klick, Doppelklick, langes Drücken)

Ein Beispiel für eine Automatisierung, die auf einen Doppelklick an Eingang 1 reagiert, legen Sie in Home Assistant am einfachsten grafisch an: **Automatisierung** → **Auslöser** → **Gerät** → **R - Control IP 4** → **Eingang 1 doppelt geklickt**.

Ab hier folgt der **vollständige Referenzteil** mit allen Details.

Diese Dokumentation beschreibt die **MQTT-Schnittstelle** des R-Control IP4 aus Integrationssicht. Sie richtet sich an Kunden/Partner, die das Gerät über einen MQTT-Broker anbinden möchten (insbesondere **Home Assistant MQTT Discovery**).

Stand: Implementierung in `components/ha_mqtt` (Firmware-Repo). Die Schnittstelle ist Broker-basiert und publish/subscribe-orientiert.

1. Überblick

Zweck

- Relaiszustände publizieren und Relais über Command-Topics schalten.
- Optional: Temperaturwert (NTC) publizieren.
- Optional: Taster-/Eingangseignisse als Home Assistant „Device Automation Triggers" publizieren.
- Home Assistant Auto-Discovery über MQTT (Discovery-Prefix konfigurierbar).

Eigenschaften

- Transport: MQTT über TCP (URI-basiert, z. B. `mqtt://<broker>:1883`)
- Authentifizierung: Broker-Login optional (`username/password`)
- QoS: 0..2 (konfigurierbar)
- Retain: konfigurierbar für State-Topics
- Availability / LWT: wird gesetzt (online/offline)

2. Aktivierung & Konfiguration

Die MQTT-Funktion wird über die Gerätekonfiguration aktiviert.

2.1 Konfigurationspfad

Alle MQTT-Settings liegen unter:

- `config.interfaces.ha_mqtt.*`

2.2 Konfigurationsschlüssel

Pflicht/Standard:

- `config.interfaces.ha_mqtt.enabled` (bool)
 - `true` = MQTT-Client wird gestartet
 - `false` = MQTT-Client wird gestoppt
- `config.interfaces.ha_mqtt.uri` (string)
 - Broker-URI, z. B. `mqtt://192.168.0.2:1883`
- `config.interfaces.ha_mqtt.username` (string, optional)
- `config.interfaces.ha_mqtt.password` (string, optional)
- `config.interfaces.ha_mqtt.base_topic` (string)
 - Default: `r-control-ip4`
- `config.interfaces.ha_mqtt.discovery_prefix` (string)
 - Default: `homeassistant`
- `config.interfaces.ha_mqtt.qos` (int 0..2)
 - Default: `1`
- `config.interfaces.ha_mqtt.retain` (bool)
 - Default: `true`
- `config.interfaces.ha_mqtt.keepalive` (int, Sekunden)
 - Bereich: `15..3600`
 - Default: `60`

Optionale Temperaturpublizierung:

- `config.interfaces.ha_mqtt.temperature.enabled` (bool)
 - Default (Firmware): `true`
- `config.interfaces.ha_mqtt.temperature.min_interval_ms` (int)
 - Default (Firmware): `10000`
 - `0` = keine Drosselung

Hinweis: Im Beispiel `spiffs/config.json` sind nur die Basisfelder enthalten. Die `temperature.*`-Werte sind optional und nutzen Firmware-Defaults, wenn nicht vorhanden.

3. Device Identity (device_id)

Die MQTT Topics enthalten eine `device_id`, die firmwareseitig automatisch gebildet wird:

- Basis ist `config.network.lan.eth0.hostname`.
- Der Hostname wird in ein MQTT/HA-kompatibles ID-Format normalisiert (Kleinbuchstaben, Sonderzeichen → `_`).
- Zusätzlich werden (wenn verfügbar) die letzten 3 Bytes der MAC-Adresse angehängt.

Beispiel (schematisch):

- Hostname: `R-Control-IP4`
- device_id: `r_control_ip4_a1b2c3`

4. Topic-Struktur

4.1 Base Topics

Mit `base_topic=<BT>` und `device_id=<ID>`:

- Device Availability (global):
 - `<BT>/<ID>/status`
 - Payload: `online` / `offline` (retained)
- Relais (1..4):
 - State: `<BT>/<ID>/relay/<n>/state`
 - Payload: `ON` / `OFF` (retained je nach `retain`)
 - Command: `<BT>/<ID>/relay/<n>/set`
 - Payload: `ON` / `OFF` / `TOGGLE`
- Button Events (1..4):
 - `<BT>/<ID>/button/<n>/event`
 - Payloads: `pressed`, `released`, `click`, `double_click`, `long_press`, `long_press_release`
 - retain: `false`
- Temperatur (optional):
 - State: `<BT>/<ID>/temperature/state`
 - Payload: Float als String, mit 2 Nachkommastellen (z. B. `22.50`)
 - Availability (sensor-spezifisch): `<BT>/<ID>/temperature/status`
 - Payload: `online` (nur wenn Sensorstatus OK) / `offline`

4.2 Home Assistant Discovery Topics

Mit `discovery_prefix=<DP>`:

- Relais als HA Switch Entities:
 - `<DP>/switch/<ID>/relay_<n>/config`

- Temperatur als HA Sensor Entity (wenn aktiviert):
 - `<DP>/sensor/<ID>/temperature/config`
- Buttons als HA Device Automation Triggers:
 - `<DP>/device_automation/<ID>/button_<n>_<event>/config`

Discovery Payloads sind JSON (unformatted), retained (`true`).

5. Payloads & Semantik

5.1 Relais Command Payload

Akzeptierte Payloads auf `<BT>/<ID>/relay/<n>/set`:

- `ON` → Relais einschalten
- `OFF` → Relais ausschalten
- `TOGGLE` → Relais toggeln

Die Firmware akzeptiert die Kommandos case-insensitiv (wird intern auf Uppercase normalisiert).

5.2 Relais State Payload

Auf `<BT>/<ID>/relay/<n>/state` publiziert die Firmware:

- `ON` oder `OFF`

Publiziert wird:

- bei MQTT-Connect initial (Snapshot)
- bei Relais-Events (State Changed / Impulse Done / Blink Tick)

5.3 Availability (LWT)

- Beim Connect publiziert das Gerät `online` auf `<BT>/<ID>/status`.
- Es ist ein Last Will gesetzt, der `offline` publiziert, falls die Verbindung unerwartet abbricht.
- LWT ist retained.

5.4 Temperaturpublizierung (optional)

- Nur wenn `config.interfaces.ha_mqtt.temperature.enabled=true`.
- Availability des Sensors wird separat geführt:
 - `online` nur bei `NTC_SENSOR_STATUS_OK`
 - sonst `offline`
- Temperaturwerte werden gedrosselt:
 - Minimum-Intervall `temperature.min_interval_ms`
 - Zusätzlich darf trotz Drosselung publiziert werden, wenn sich der Wert um mindestens `0.2°C` ändert.

6. Integrationshinweise (Best Practices)

6.1 Topic-Design

- Für Custom-Integrationen empfiehlt sich, nur die State/Command-Topics zu verwenden.
- Für Home Assistant ist MQTT Discovery (Discovery Prefix) der bevorzugte Weg.

6.2 Idempotenz

- Idempotent: **ON**, **OFF**
- Nicht-idempotent: **TOGGLE** (Retries können unerwartete Zustandswechsel erzeugen)

Empfehlung: In robusten Systemen **TOGGLE** vermeiden und deterministisch **ON/OFF** senden.

6.3 QoS/Retain

- QoS **1** ist ein guter Default für LAN-Integrationen.
- Retain **true** für Relais-State ist sinnvoll, damit neue Subscriber sofort den letzten Zustand erhalten.
- Button-Events sind bewusst nicht retained.

7. Security-Hinweise

Sicherheit hängt maßgeblich vom MQTT-Broker ab.

Empfehlungen:

- Broker-Authentifizierung aktivieren (User/Pass) und Rechte pro Client einschränken (ACLs).
- Broker nur im vertrauenswürdigen Netz betreiben oder TLS/**mqtt**s:// verwenden (abhängig vom Broker/Setup).
- Topics so designen/ACLs so setzen, dass nur autorisierte Systeme auf **.../relay/<n>/set** publishen dürfen.

8. Beispiele

8.1 Beispiel (mosquitto_sub)

```
# Alle Topics des Geräts beobachten
mosquitto_sub -h 192.168.0.2 -t 'r-control-ip4/#' -v

# Discovery beobachten
mosquitto_sub -h 192.168.0.2 -t 'homeassistant/#' -v
```

8.2 Beispiel (mosquitto_pub)

```
# Relais 1 einschalten
mosquitto_pub -h 192.168.0.2 -t 'r-control-ip4/<device_id>/relay/1/set' -m 'ON'

# Relais 1 ausschalten
mosquitto_pub -h 192.168.0.2 -t 'r-control-ip4/<device_id>/relay/1/set' -m 'OFF'
```

Tipp: `<device_id>` kann man im Broker leicht finden, indem man zunächst `mosquitto_sub -t 'r-control-ip4/+/status' -v` ausführt.

9. Kompatibilität & Änderungen

Die MQTT-Integration ist Home-Assistant-kompatibel (Discovery) und kann in zukünftigen Firmware-Versionen erweitert werden.

Für Integrationen mit sehr spezifischen Anforderungen (z. B. komplexe Zustandsmodelle) können alternativ die JSON-REST-API oder das WebSocket-Gateway genutzt werden (siehe [docs/API.md](#)).